

Complementos de Investigação Operacional

5.2 Programação dinâmica: o problema da mochila

José Firmino Aguilar Madeira

O problema da mochila

Há uma capacidade limitada W e vários tipos de objetos, cada um com peso w_i e valor v_i . Que objetos levar para **maximizar o valor**, sem exceder a capacidade?

- **Mochila ilimitada**: pode levar-se qualquer número de unidades de cada tipo.
- **Mochila 0–1**: cada objeto leva-se ou não (uma unidade).

Primeira ideia: escolher pelo maior valor por unidade de peso, v_i/w_i . Funciona às vezes — mas não sempre.

Onde aparece

- carga de camiões, contentores, aviões;
- seleção de projetos com orçamento limitado;
- corte de material (bobinas, chapas);
- escolha de anúncios para um intervalo publicitário;
- alocação de memória e de largura de banda.

É um problema de otimização inteira; a PD resolve-o com uma tabela.

Exemplo 1: carregar um camião (mochila ilimitada)

Um camião leva até 10 t. Há três tipos de paletes, em número suficiente:

Paleta	w_i (t)	v_i (centenas €)	v_i/w_i
A	3	7	2,33
B	4	13	3,25
C	6	17	2,83

Pelo rácio: B, B (8 t, lucro 26); sobram 2 t, onde não cabe nada.

Recursão

$f(k)$ = lucro máximo com capacidade k :

$$f(k) = \max_{i: w_i \leq k} \{v_i + f(k - w_i)\},$$

com $f(k) = 0$ se nada cabe.

Estado = capacidade livre k ; decisão = que paleta pôr a seguir. Calcula-se $f(0), f(1), \dots, f(10)$ por esta ordem: cada valor usa valores já calculados.

A tabela da mochila ilimitada

k	A: $7 + f(k - 3)$	B: $13 + f(k - 4)$	C: $17 + f(k - 6)$	$f(k)$	melhor
0-2	—	—	—	0	—
3	7	—	—	7	A
4	7	13	—	13	B
5	7	13	—	13	B
6	14	13	17	17	C
7	20	20	17	20	A ou B
8	20	26	17	26	B
9	24	26	24	26	B
10	27	30	30	30	B ou C

Ler a solução a partir de $k = 10$: $f(10) = 13 + f(6) \rightarrow$ pôr B, restam 6 t; $f(6) = 17 + f(0) \rightarrow$ pôr C, restam 0. (A coluna C dá o mesmo: C e depois B.)

Resultado

Uma paleta B e uma C: 10 t, lucro 30 — contra 26 pelo rácio. Por força bruta, só esta carga atinge 30.

Exemplo 2: escolher projetos com orçamento (mochila 0–1)

Uma câmara municipal tem 10 (centenas de milhares de €) para investir e cinco projetos candidatos; cada um faz-se ou não.

Projeto	Custo w_i	Benefício v_i	v_i/w_i
P1 ciclovias	3	12	4,0
P2 piscina	5	14	2,8
P3 iluminação LED	3	15	5,0
P4 painéis solares	4	15	3,75
P5 digitalização	1	8	8,0

Benefício: valor atual líquido estimado.

Pelo rácio: P5, P3, P1 (custo 7, benefício 35); P4 e P2 já não cabem.

Formulação por etapas

Etapa i = projeto i ; estado k = orçamento ainda disponível; decisão $x_i \in \{0, 1\}$.

$$f_i(k) = \max \left\{ \underbrace{f_{i+1}(k)}_{\text{não}}, \underbrace{v_i + f_{i+1}(k - w_i)}_{\text{sim, se } w_i \leq k} \right\}$$

com $f_6(k) = 0$. Resolve-se de P5 para P1.

A tabela da mochila 0–1

orçamento k	0	1	2	3	4	5	6	7	8	9	10
f_5 (só P5)	0	8	8	8	8	8	8	8	8	8	8
f_4 (P4, P5)	0	8	8	8	15	23	23	23	23	23	23
f_3 (P3–P5)	0	8	8	15	23	23	23	30	38	38	38
f_2 (P2–P5)	0	8	8	15	23	23	23	30	38	38	38
f_1 (todos)	0	8	8	15	23	23	27	35	38	38	42

Ler a solução ($k = 10$):

- P1: não $f_2(10) = 38$, sim $12 + f_2(7) = 42 \rightarrow$ **sim**;
restam 7.
- P2: não $f_3(7) = 30$, sim $14 + f_3(2) = 22 \rightarrow$ não.
- P3: não $f_4(7) = 23$, sim $15 + f_4(4) = 30 \rightarrow$ **sim**;
restam 4.
- P4: não $f_5(4) = 8$, sim $15 + f_5(0) = 15 \rightarrow$ **sim**;
restam 0.
- P5: não cabe.

Resultado

P1, P3 e P4: custo 10, benefício **42** — contra 35 pelo rácio.

Repare: o projeto mais “eficiente” (P5) fica de fora — gastaria 1 do orçamento que falta para P4.

Ilimitada ou 0–1: o que muda

	Ilimitada	0–1
Estado	capacidade livre k	(objeto i , capacidade livre k)
Recursão	$f(k) = \max_i \{v_i + f(k - w_i)\}$	$f_i(k) = \max \{f_{i+1}(k), v_i + f_{i+1}(k - w_i)\}$
Tabela	uma linha ($W + 1$ valores)	uma linha por objeto ($n \times (W + 1)$)
Esforço	cerca de $n W$ somas	cerca de $n W$ somas

- Na 0–1 é preciso lembrar **que objetos já foram usados**: por isso o estado inclui a etapa i .
- O esforço cresce com W : com capacidades muito grandes (ou pesos com casas decimais) a tabela fica enorme. Aí usam-se *branch and bound* (cap. 3) ou solvers de programação linear inteira.

Em Python, e a confirmação por programação linear inteira

```
def mochila_01(w, v, W):
    n = len(w)
    F = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(n - 1, -1, -1):          # P5 -> P1
        for k in range(W + 1):
            F[i][k] = F[i + 1][k]          # não
            if w[i] <= k:                  # sim
                F[i][k] = max(F[i][k],
                               v[i] + F[i + 1][k - w[i]])

    x, k = [], W
    for i in range(n):
        x.append(int(F[i][k] != F[i + 1][k]))
        k -= w[i] * x[-1]
    return F[0][W], x

print(mochila_01([3, 5, 3, 4, 1], [12, 14, 15, 15, 8], 10))
# (42, [1, 0, 1, 1, 0])
```

```
from scipy.optimize import milp, LinearConstraint
import numpy as np
w = np.array([3, 5, 3, 4, 1])
v = np.array([12, 14, 15, 15, 8])
r = milp(-v,
          constraints=LinearConstraint(w, 0, 10),
          integrality=np.ones(5), bounds=(0, 1))
print(-r.fun, r.x)      # 42.0 [1. 0. 1. 1. 0.]
```

A mesma resposta pelos dois métodos. Na mochila ilimitada basta trocar bounds=(0, 1) por (0, np.inf).

Exemplo 4 — o navio

Um navio com capacidade de 4 t pode transportar um ou mais produtos da tabela (w_i em toneladas; R_i , receita unitária em milhares de €). Determine o número de unidades de cada produto que maximiza a receita.

Produto	w_i	R_i
1	2	31
2	3	47
3	1	14

Exemplos 5 e 6 — mochila (várias unidades de cada tipo)

Ex. 5: 10 kg			Ex. 6: 13 kg		
Prod.	Peso	Valor	Prod.	Peso	Valor
A	3	6	A	3	6
B	4	7	B	4	7
C	7	10	C	5	10
			D	6	12

Quais e quantos produtos colocar na mochila para maximizar o valor?

O QUE DEVE FICAR — 5.2

1. Mochila ilimitada: $f(k) = \max_i \{v_i + f(k - w_i)\}$, para $k = 0, \dots, W$.
2. Mochila 0–1: etapa = objeto, estado = capacidade livre;
 $f_i(k) = \max\{f_{i+1}(k), v_i + f_{i+1}(k - w_i)\}$.
3. A solução lê-se para trás na tabela, a partir de $f(W)$ (ou $f_1(W)$).
4. Escolher pelo rácio v_i/w_i pode falhar (26 contra 30; 35 contra 42).
5. Confirmar com força bruta (poucos objetos) ou com programação linear inteira.

A seguir: 5.3 Gestão da mão-de-obra

Referências Lista completa: *Bibliografia da UC*.

- Hillier, F. S., Lieberman, G. J., *Introduction to Operations Research*, 11.ª ed., McGraw-Hill, 2021 — Dynamic Programming.
- Kellerer, H., Pferschy, U., Pisinger, D., *Knapsack Problems*, Springer, 2004.
- Taha, H. A., *Operations Research: An Introduction*, 10.ª ed., Pearson, 2017 — o modelo da carga (*knapsack*).

Recursos

- Lista de exercícios 5. `code/python/05_dynamic_programming/05_2_knapsack/mochila.py`.